

Distributed Systems

Leader Election



CUHK

Leader Election

- Very useful because once we have the leader
 - The leader makes the decision and everyone follows
 - Reduce the n^2 distributed setting back to the 1-to-n setting
 - E.g., Raft first elects a leader for its replicated state machine consensus problem
 - Become a Primary-replica replication problem
 - From n^2 peer replications To only leader serves client and replicate to others
- In centralized model
 - the leader dies => the system dies
- In distributed, leader election is the problem of once a leader dies (e.g., not sending heartbeat to the others), the nodes elect a new leader that is agreed by everybody as soon as possible

Leader Election vs Mutual Exclusion

- In some sense, it is also like an ME problem because every time only ONE guy can take the lock (take the “leader role”)
- So, one might wonder we can use a distributed ME algorithm to solve LE?
- No:
 - ME algorithms don't deal with failed leader
 - ME hates starvation
 - Vs. LE: A leader can stay as long as there is no failure
 - i.e., it can stay in the CS as long as it likes (so it violates ME's bounded waiting)

Leader Election

- There is no politics indeed
 - Given me **one unique process as the leader ASAP**
- Each process has
 - a unique ID
 - a variable L; the id of my leader
- V: set of IDs
 - $\forall i, j \in V : i, j \text{ are nonfaulty} :: L(i) \in V \quad \wedge \quad \mathbf{L(i) = L(j)} \quad \wedge \quad L(i) \text{ is nonfaulty}$
agree on same leader

Difficulty of implementing and testing – Leader Election

- Case 1:

- Both A and B detect Leader L dies
- They both publish that they want to be the leader
- For C and D, who should they vote for?
 - Vote for the one who asks for your vote first?
 - What if A's request arrives C first but B's request arrives D first? **Split vote**

- Case 2:

- A, B, and C vote for A be the new leader and while A is doing the following:
 - While {for each other node i}
 - Send '*I am the new leader, yeah*' to i. **//Then L wakes up and sends heartbeat again...**

- Case 3:

- While A is sending out the winning message to B, before sending to C, A dies....
- ... “enjoy” more of them in your Raft part 1

What about this 1-line protocol?

- Given a set of nodes with IP1, IP2, IP3, IP4, IP5
 - *We set the one with the largest IP as the leader*
- Now, IP5 leader is dead
- Others think IP4 is the new leader
- But what if IP5 sends heartbeats again?
- So, is this protocol complete in terms of safety and liveness?
- Bully algorithm makes this idea complete under a not-so-real system model

Bully Algorithm [Garcia-Molina82]

- Goal: Elect the one with the highest ID as the leader
- Assume links are fault-free
- Assume failure mode is fail-stop (next topic) and can be detected by time-out (=sync)
- Once a process detects a leader has failed (e.g., by time-out)
 - That process will trigger Bully Algorithm to begin the election
 - Eventually, the non-faulty process with the highest ID is the new leader



3 kinds of messages:

- Election; Reply; Leader

Any process who detects failure,

- will attempt to become the new leader by
- sending an <Election> msg to processes ~~es~~ with ID higher than it
 - Essentially implying that “Can I be the new leader?”
 - Why not send only to the process with the next highest ID?
 - Hector’s design choice: because that process may also die

If any process (with a higher ID) replies, then the requesting process gives up and wait for a <Leader> msg to follow the new leader

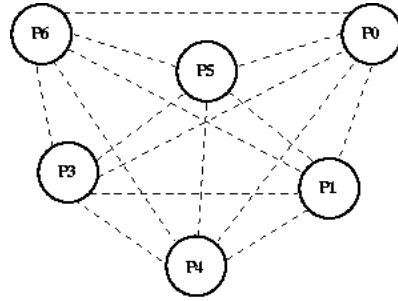
If I receive nobody replying my <Election> message within t sec, then I think I can be the leader and send a <Leader> message to all others to announce my winning

If someone turns down my <Election> but I receive no <Leader> message from him after time-out, I assume the new leader also dies and I re-trigger the process

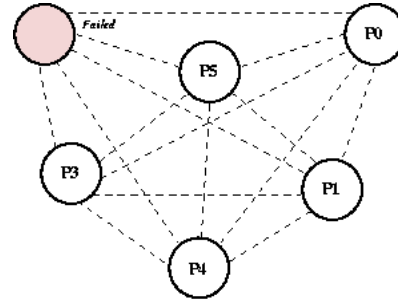
If I receive an <Election> message from another process with a lower ID,

- I <Reply> back
- I **restart** the election process by sending an <Election> to higher-numbered processes.

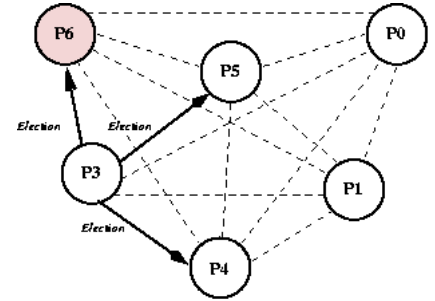
Example



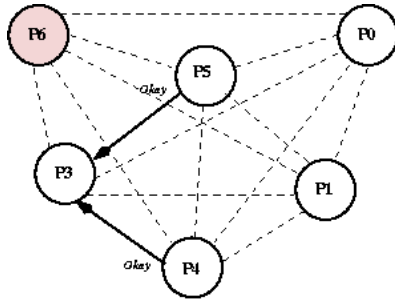
Bully Algorithm: Step 0



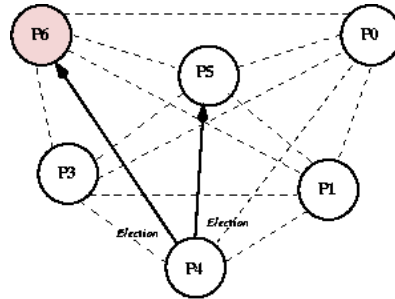
Bully Algorithm: Step 1



Bully Algorithm: Step 2

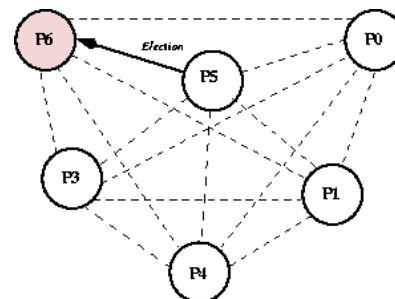


Bully Algorithm: Step 3

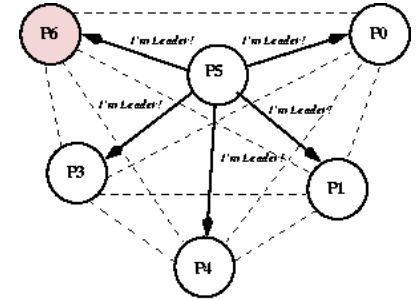


Bully Algorithm: Step 4

...



Bully Algorithm: Step 6



Bully Algorithm: Step 7

Message Complexity

- Worst-case happens when the process with the lowest-id (detects current leader fail) W_0 and triggers the election
 - Best-case is the process with the highest-id triggers
- W_0 sends $n-1$ <election> messages to other processes with higher IDs
- All receiving processes turn that down and each independently triggers a new election and so
 - W_1 : sends $N-2$ <election> messages to $W_2, W_3, \dots$
 - W_2 : sends $N-3$ <election> messages to $W_3, W_4, \dots$
 - ...
 - $W_{\max}$: sends to myself
 - $O(N^2)$ <election> messages
 - $O(N^2)$ <reply> messages
 - $O(N)$ <leader> messages (to notify them the winner)
 - $\rightarrow O(N^2)$ complexity

Other logical/physical network topology?

- Bully works on fully connected synchronous network only
 - Meaning: messages from X to $\{w, h, a, t, e, v, e, r\}$ arrive roughly the same time
- Other types of network. E.g.,
 - ring network, each node only knows two entries
 - $\langle \text{left-neighbor, IP-address} \rangle$
 $\langle \text{right-neighbor, IP-address} \rangle$
 - tree network, often used in a small data center

